# VERTICAL SCALING VS HORIZONTAL SCALING

SUDHEESH A R

#  VERTICAL SCALING (SCALING UP):

- **Definition:** Adding more power (CPU, RAM, SSD) to your existing server.
- **Pros:**
  - Easier to implement (just upgrade the machine)
  - Less complex architecture.
- **Cons:**
  - Limited by the capacity of a single machine.
  - Downtime is often required for upgrades.
  - More expensive per unit of resource.

# HORIZONTAL SCALING (SCALING OUT):

- **Definition:** Adding more machines/nodes to distribute the load.
- Pros:
  - Almost infinite scalability (just add more nodes!).
  - Improved fault tolerance (if one node fails, others pick up the slack).
  - Supports distributed systems (microservices, clusters, etc.).
- Cons:
  - More complex to design (load balancers, distributed databases, etc.).
  - Requires managing consistency, synchronization, etc.
  - Higher upfront infrastructure cost.

# COMPARISON

| Aspect | Vertical Scaling | Horizontal Scaling |
| --- | --- | --- |
| **Capacity Limit** | Limited by hardware | Almost unlimited (add more nodes) |
| **Cost** | Cheaper initially, expensive at high-end | More upfront, cost-effective long-term |
| **Fault Tolerance** | Single point of failure | High (redundancy) |
| **Complexity** | Simple to implement | More complex (distributed systems) |
| **Downtime** | Often needed | Zero-downtime scaling possible |

# USAGE

| Use Case | Best Approach |
| --- | --- |
| Monolithic legacy app, low traffic | Vertical scaling |
| Real-time distributed systems (microservices) | Horizontal scaling |
| Need quick performance boost | Vertical scaling |
| Long-term scalability & reliability (cloud-based) | Horizontal scaling |

# OTHER TYPES

- Diagonal Scaling (Combination of Vertical + Horizontal)
  - First scale vertically (upgrade resources).
  - Then when maximum vertical scaling is reached, start scaling horizontally by adding nodes.

Example:

You first make your server bigger (vertical), and when it's not enough, you add more servers (horizontal).

👉 This is very common in real-world cloud setups (AWS, Azure, GCP).

# AUTO SCALING

- Automatic scaling based on system load.
- It can scale up/down (vertically) or scale out/in (horizontally) based on CPU, memory, network, or custom metrics.
- Example:
  In AWS, Auto Scaling Groups automatically add/remove EC2 instances based on traffic load.

# GEOGRAPHIC SCALING (GEO-SCALING / MULTI-REGION SCALING)

- Deploy services across **multiple geographic regions** to reduce latency and increase availability.
- Handle user requests closer to their location.
- **Example:**
  Netflix serves users from nearby regional data centers to ensure fast video streaming.

# FUNCTIONAL SCALING

- Scale based on **features or functionality**.
- Different parts of the application are scaled independently depending on their usage patterns.
- **Example:**
  **You separately scale your search service and your user authentication service because they have different loads.**

# ELASTIC SCALING

- Dynamically and automatically **scale resources both up and down** (add or remove resources **on-demand**) based on real-time needs.
- Elastic scaling is the **core principle of cloud-native applications**.

| Aspect | Auto Scaling | Elastic Scaling |
| --- | --- | --- |
| Triggered by | Predefined rules | Real-time demand |
| Human setup needed? | Yes (set rules/thresholds) | Minimal (system handles it automatically) |
| Typical example | Adding EC2 instances in AWS | AWS Lambda scaling with incoming requests |
| Type of services | VMs, Containers | Serverless, PaaS |
| Responsiveness | Reactive (based on thresholds) | Proactive (based on real-time load) |

# SUMMARY

| Type | Description | Example |
| --- | --- | --- |
| Vertical Scaling | Add resources to the same server | Increase RAM/CPU of a VM |
| Horizontal Scaling | Add more servers/nodes | Add more web servers |
| Diagonal Scaling | Mix of vertical and horizontal | Scale up first, then scale out |
| Auto Scaling | Scale automatically based on metrics | AWS Auto Scaling Group |
| Geographic Scaling | Distribute systems across multiple locations | Netflix regional servers |
| Functional Scaling | Scale components independently | Scale login and search separately |
| Elastic Scaling | Dynamic scaling up/down based on real-time demand | AWS Lambda, Azure Functions |